CryptDNN: A Fast Privacy-Preserving Deep Neural Network Inference Architecture Based on Cloud-Edge-Client Collaboration

Dong Li, *Graduate Student Member, IEEE*, Anupam Chattopadhyay, *Senior Member, IEEE*, Qianyu Li, Qingguo Lü, *Member, IEEE*, Jiahui Wu, Tao Xiang, *Senior Member, IEEE*, and Xiaofeng Liao, *Fellow, IEEE*

***Abstract*—Deep neural networks (DNNs) with fully homomorphic encryption (FHE) are gaining traction in Internet of Things (IoT) applications requiring robust protection of data inference and model privacy. However, the existing implementations typically replace activation functions with approximate polynomial functions, which compromises the accuracy and efficiency of machine learning. To overcome these limitations, we introduce CryptDNN, a fast privacy-preserving DNN inference architecture based on cloud-edge-client collaboration. In CryptDNN, the DNN model and the data in IoT are deployed in the cloud server, the client, and the edge server in a distributed manner. Specifically, we first encrypt the client's data by using FHE, then divide the DNN into two components: a linear module encrypted with FHE which is stored in the cloud server, and a non-linear module represented as an encrypted look-up table via searchable symmetric encryption (SSE), which is stored in the edge server, thus avoid the need for low-degree polynomial approximations of the activation function under ciphertext environments. The security analysis confirms that the CryptDNN robustly safeguards the confidentiality of both the data and the model. Experimental results on the realistic MNIST and CIFAR-10 datasets also demonstrate that the CryptDNN achieves high inference accuracy and enhanced computational efficiency.**

***Index Terms*—Deep Neural Network, fully homomorphic encryption, searchable symmetric encryption, look-up table, privacy-preserving.**

I. Introduction

IN RECENT years, the seamless integration of cloud services and machine learning has given rise to a novel service known as Machine Learning as a Service (MLaaS) [1], [2], [3]. In MLaaS, cloud service providers enable clients to train and deploy DNN models on their cloud servers. Once deployed, clients can utilize these models for various predictive tasks. Nowadays, MLaaS has found widespread application across various IoT domains such as healthcare [4], finance [5], business [6]. In particular, recent studies have explored the integration of MLaaS into advanced IoT-enabled infrastructures, including vehicular edge computing [7], UAV-assisted IoT networks [8], and non-linear edge systems [9], further demonstrating the versatility and scalability of MLaaS in real-world intelligent environments.

Unfortunately, MLaaS deployed in IoT fails to ensure the confidentiality of the client's data and the model parameters. For instance, when clients input raw data from smart sensor devices into a pre-trained deep neural network (DNN) model for predictive tasks, there is a risk of data exposure—particularly in sensitive domains such as medical diagnosis, where the data may contain personal health information. If adversaries manipulate the model parameters, it could result in incorrect diagnoses. The potential compromise of such data poses significant risks to the client, highlighting the urgent need for a robust privacy-preserving MLaaS framework in the IoT ecosystem.

Cryptography-based methods, such as fully homomorphic encryption (FHE) [10], searchable symmetric encryption (SSE) [11], and secure multi-party computation (SMC) [12], have emerged as promising approaches to address privacy concerns in machine learning as a service (MLaaS). For example, Dowlin et al. [13] proposed CryptoNets, a secure inference framework based on the YASHE[1] [14] scheme. However, CryptoNets supports only a limited number of non-linear layers due to constraints on multiplicative depth, rendering it unsuitable for deeper convolutional neural networks (CNNs). Later, Chabanne et al. [15] introduced batch normalization into the CryptoNets framework to support multiple non-linear layers. Despite these advances, a fundamental trade-off remains between computational efficiency and inference accuracy, especially when deeper architectures and non-linear activation functions are involved.

Received 22 January 2025; revised 7 September 2025; accepted 6 October 2025. Date of publication 13 October 2025; date of current version 11 December 2025. This work was supported in part by the Graduate Research and Innovation Foundation of Chongqing under Grant CYB25038, in part by the Natural Science Foundation of Chongqing (Innovation and Development Joint Fund) under Grant CSTB2025NSCQ-LZX0085 and Grant CSTB2023NSCQ-LZX0149, and in part by the National Natural Science Foundation of China under Grant U20A20176, Grant 62576053, and Grant 62302068. Recommended for acceptance by Prof. Geng Sun. *(Corresponding authors: Qingguo Lü; Xiaofeng Liao.)*

Dong Li, Qingguo Lü, Tao Xiang, and Xiaofeng Liao are with the Key Laboratory of Dependable Services Computing in Cyber Physical & Society-Ministry of Education, College of Computer, Chongqing University, Chongqing 400044, China (e-mail: lilvmy@163.com; lvqingguo886@126.com; txiang@cqu.edu.cn; xfliao@cqu.edu.cn).

Anupam Chattopadhyay is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: anupam@ntu.edu.sg).

Qianyu Li is with the College of Electronic Engineering, National University of Defense Technology, Hefei 230000, China (e-mail: liqianyu@nudt.edu.cn).

Jiahui Wu is with Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: jiahuiwu2022@163.com).

Digital Object Identifier 10.1109/TNSE.2025.3620373

[1] Yet another somewhat homomorphic encryption scheme.

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

To overcome these limitations, we introduce CryptDNN, a fast privacy-preserving DNN inference architecture based on cloud-edge-client collaboration that maintains the inference accuracy of plaintext DNN models. Specifically, CryptDNN adopts a novel distributed inference paradigm in which the client, the cloud server (called computation platform), and the edge storage server (called storage platform) jointly participate in the encrypted inference process. This design reduces the multiplicative depth required by the underlying Brakerski-Fan-Vercauteren (BFV) scheme [16], [17], thereby improving computational efficiency and scalability. By partitioning the DNN into linear and non-linear modules, where linear layers such as convolutional, pooling and fully connected layers are processed homomorphically on the cloud server and non-linear activation layers are executed on the edge storage server using the proposed encrypted look-up table, thus CryptDNN supports deeper models while preserving exact ReLU functionality without approximation. Furthermore, to enhance the security of non-linear operations, CryptDNN incorporates a Pseudo-Random Function (PRF) [18]-based searchable symmetric encryption scheme to transform activation computation into secure ciphertext search, which significantly reduces inference latency while protecting intermediate data privacy. The main contributions of this paper are as follows.

- Unlike the existing approaches that rely on FHE, SMC, or secret sharing, the CryptDNN proposes an integrated approach by combining FHE with SSE. This is to uphold the confidentiality of both client data and DNN model parameters, where FHE is used to encrypt the data and the model, SSE is used to construct an encryption look-up table for ReLU function.
- The CryptDNN employs a divide-and-conquer approach and separates the DNN model into linear and non-linear modules. This strategic operation addresses the challenges associated with using approximate functions or Taylor expansion in place of ReLU under ciphertext conditions, which often leads to the reduction of model accuracy and the increase of communication costs due to the utilization of garbled circuits. By doing so, CryptDNN effectively enhances both the speed and accuracy of model inference.
- The experimental results on the MNIST datasets demonstrate that CryptDNN achieves an accuracy of 98.92%, which is remarkably close to the accuracy of the plaintext model, standing at 98.97%. Furthermore, the time required for predicting an encrypted image is 89.501 seconds, significantly lower than the alternative method of using a square function in place of ReLU, which takes 185.62 seconds. These results once again confirm that CryptDNN provides efficient, accurate, and scalable privacy-preserving predictions.

The rest of this paper is organized as follows. We introduce related work on privacy-preserving neural networks in Section II, while providing brief descriptions of BFV encryption scheme and DNN in Section III. The architecture and concrete construction of the CryptDNN are presented in Sections IV and V, respectively. Additionally, we provide detailed security analysis, adaptability analysis, and performance evaluation of the CryptDNN in Sections VI and VII, respectively. Finally, we conclude the CryptDNN in Section VIII.

II. Related Work

In privacy-preserving DNN inference fields, a plethora of works have captured considerable attention and these works can be categorized into two main groups: some leveraging trusted execution environments (TEEs) [19], [20], and others employing cryptographic methods, such as SMC and FHE. The TEE-based approaches follow a unique trajectory, offering security assurances contingent upon trust in TEE providers. Hence, we will not delve into this avenue here, opting instead to study the cryptographic-based secure inference methodologies. In Table I, we have roughly compared the existing privacy-preserving DNN inference schemes.

FHE-Based Schemes: The CryptoNets [13] stands out as the pioneering privacy-preserving convolutional neural network (CNN) scheme by employing FHE, and innovatively substituting the quadratic function for these non-linear activation functions to address the issue that FHE cannot compute the non-linear function. Based on CryptoNets, Falcetta et al. [21] adopted the BFV homomorphic encryption scheme and polynomial approximation to be compatible with the LeNet model and got 98.26% correct results in the MNIST datasets. However, their scheme has undergone a slight change for the LeNet model by reducing the activation layer number to obtain higher accuracy. To address this problem, prior works utilized low-degree polynomial approximation to replace the activation function, then adopted the FHE to encrypt the data and obtained the final encryption outputs. For example, CHET [22] introduced an optimizing compiler that is capable of autonomously optimizing parameter choices for secure inference to enhance performance. Yang et al. [23] and Lu et al. [24] used FHE technology to design a vertical cooperative learning scheme for neural networks to resist collusion attacks. In their scheme, split learning and FHE technologies are combined to achieve ciphertext inference. Manto [25] facilitated accelerated cryptographic inference for clients while preserving inference accuracy. This was achieved by employing various quadratic activation functions to substitute for specific ReLU layers, coupled with sliding-window-based fine-tuning techniques to generate CNN models with minimal or no ReLUs. Recent advances further reduce rotation/bootstrapping costs and improve packing efficiency for CNNs and Transformers [27], [28], [29], [30], [33], [34], [35], [36], [37], [38] and GPU-accelerated HE kernels/systems have emerged to speed up homomorphic encryption operations [31], [32]. For example, Kim et al. [27] proposed an optimized privacy-preserving CNN inference scheme using FHE, achieving lower latency and communication overhead but remaining limited to convolutional models. HyPHEN [28] introduced a hybrid packing method that improves the efficiency of HE-based neural networks through flexible parallelism, though it requires careful tuning for different architectures. Lee et al. [30] designed multiplexed parallel convolutions to enable low-complexity CNNs over FHE, significantly reducing computational costs but with limited applicability

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING PRIVACY-PRESERVING DNN INFERENCE SCHEMES

Schemes	*Linear Layer Scheme*	*Non-Linear Layer Scheme*	*Inference Overhead*	*Communication Overhead*
CryptoNets [13], SecureSL [23], CHET [22]	FHE	FHE	High	Low
SecureML [46], Delphi [47]	AHE+ASS	GC	High	High
MiniONN [48], CryptoNAS [49]	AHE	GC	High	Low
Cheetah [50]	FHE+ASS+OT	OT	High	High
SIRNN [51]	ASS+OT	OT	Low	High
PCNNCEC [53]	ASS	ASS	Low	High
PCNNHM [54]	ASS+BSS	ASS+BSS	Low	High
Manto [25]	AHE+ASS	ASS+BSS	High	High
BfvDNN [21]	FHE	FHE	High	Low
HyPHEN [28]	FHE	FHE	High	Low
Primer [38]	FHE	GC	High	High
Mpcvit [65]	ASS+OT	GC	Low	High
TfheDNN [26]	TFHE	TFHE	High	Low
PPFL [39]	MHE	ASS	High	High
SecureTrain [40]	FHE	HSS	High	High
CryptDNN	**FHE**	**SSE**	**Low**	**Low**

Notes: MHE denotes the multiplication homomorphic encryption, AHE is the additive homomorphic encryption, ASS is the additive secret sharing; BSS is the boolean secret sharing and GC is the garbled circuits, OT is the oblivious transfer protocols, HSS is the homomorphic secret share.

beyond convolutional models. Primer [38] presented a fast private Transformer inference framework on encrypted data, offering reduced latency and communication while still facing scalability challenges for long-sequence tasks. Unfortunately, these schemes adopted approximated polynomial functions to replace the activation functions, resulting in lower inference accuracy. To eliminate this defect, Stoian et al. [26] used TFHE to design a novel privacy-preserving DNN inference framework without approximated operation for the non-linear layer. Likewise, Zhang et al. [39] proposed a privacy-preserving medical image diagnosis model by using multiplication homomorphic encryption and additive secret sharing. In their scheme, they adopted a weighted average algorithm to replace the traditional sum of weight computation, thus speeding up the inference efficiency. However, the FHE in their scheme was used to encrypt partial parameters of the medical image diagnosis model, hence it cannot guarantee the confidentiality of the model parameters. Xu et al. [40] presented SecureTrain, a privacy-preserved deep learning framework that enhanced accuracy, stability, and efficiency by eliminating function approximation and reducing the permutation operation. It used homomorphic secret share and FHE for efficient computations and minimized communication costs, achieving performance comparable to plaintext models. To speed up the inference efficiency, Ali et al. [41] designed a secure hybrid deep learning model for the industrial Internet of medical things with FHE and blockchain. However, their scheme could leak the parameters of the pre-trained model. Hu et al. [42] presented a lightweight, privacy-preserving IoT classification scheme using k-means and the paillier cryptosystem, reducing key management and overheads while ensuring no data leaks. Furthermore, some works [43], [44], [45] combined FHE and federated learning technologies to achieve DNN inference with high efficiency.

Although these schemes may reduce inference overhead, they typically rely on polynomial approximations to ensure compatibility with FHE, which often leads to degraded inference accuracy. In contrast, CryptDNN derived a different approach by using an encrypted look-up table to process non-linear layers, achieving inference accuracy comparable to that of plaintext inference.

SMC-Based Schemes: Considering that FHE-based DNN inference schemes spend too much computation costs, some researchers have started using SMC to substitute for FHE to reduce computation complexity. SecureML [46] and Delphi [47] aimed to strike a balance between speed and accuracy by replacing ReLU activation with quadratic approximations and designing a planner to automate the determination of the number and replacement of these approximations. MiniONN [48] and CryptoNAS [49] utilized additive homomorphic encryption for linear layer evaluation and employed garbled circuits for non-linear layers. Likewise, Cheetah [50] used oblivious transfer protocol to replace the garbled circuits, aiming to address the difficulty of designing circuits. However, the schemes mentioned above still used the HE technique on the linear layers, and only adopted garbled circuits to achieve secure plaintext comparison for non-linear layers, which still face higher computation complexity and communication costs. To avoid adopting the FHE technique, SIRNN [51] achieved state-of-the-art inference latency and accuracy by employing secret sharing-based subprotocols and oblivious transfer protocols for non-linear operations, albeit at the expense of high round complexity, placing stringent demands on communication conditions. PDLCD [52] used discrete wavelet transform, chaos, and secure multi-party computation technologies to develop a secure cancer diagnosis model. They achieve a higher accuracy of cancer diagnosis, but they need to decrypt the data when running the inference operations, resulting in data leakage issues. PCNNCEC [53] designed a new privacy-preserving CNN inference framework with cloud-edge-client collaboration by only using additive secret sharing. Specifically, they proposed a secure two-party comparison protocol for the non-linear layer and delegated some computation tasks to the edge servers to reduce the computation costs. However, their scheme relies on frequent data exchanges between the edge servers, it could incur high communication costs. PCNNHM [54] proposed an efficient and secure CNN inference scheme for health monitoring. In their scheme, additive secret sharing and boolean secret sharing are used to guarantee the confidentiality of both the client data and the model parameters, then they deployed two edge-cloud servers to run the operations of the CNN. Even though they reduced the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

rounds of interaction between two edge-cloud servers, the communication costs are still higher. Moreover, several works [55], [56], [57], [58], [59] designed a secure federated learning scheme for model training with lightweight computation to guarantee the confidentiality of sensitive information. Recently, SMC-based systems further optimize offline/online separation, non-linear protocols, and floating-point support, enabling larger models and ViT/GPT-like architectures [60], [61], [62], [63], [64], [65], [66], [67], [68], [69], [70], [71]. There also appear frameworks targeting malicious security and GPU acceleration under practical networks [60], [62], [72], [73]. For example, Mpcvit [65] presented a vision transformer architecture optimized for MPC with heterogeneous attention, achieving strong accuracy-efficiency tradeoffs but remaining specialized for vision tasks. Liu et al. [61] proposed scalable MPC protocols for machine learning in the honest-majority setting, offering improved efficiency and practicality but relying on relatively weaker adversary assumptions. MD-ML [62] introduced a high-speed privacy-preserving machine learning framework secure against a dishonest majority, providing strong robustness but with higher computational complexity. InferDPT [69] developed a privacy-preserving inference system for closed-box large language models, enabling secure deployment of LLMs but incurring significant overhead on large-scale inference.

However, these schemes based on SMC typically require interaction between the client and the cloud server at each layer of the encrypted DNN model, resulting in substantial communication overhead and increased latency. In contrast, CryptDNN only requires interaction during the processing of non-linear layers by searching the encrypted look-up table, while all linear layers can be executed entirely on the cloud server over encrypted data without any communication. This selective interaction strategy significantly reduces the overall communication cost, particularly in deep networks where linear layers dominate the computation.

III. Preliminaries

In this section, we primarily revisit the preliminaries utilized in CryptDNN, focusing on the BFV homomorphic encryption scheme, SSE technology and deep neural network.

A. BFV Homomorphic Encryption

The BFV scheme is a level homomorphic encryption (LHE), which supports the predetermined number of homomorphic operations and the security depends on the Ring-Learning With Errors (RLWE) problem. The construction of the BFV scheme is mainly divided into 6 parts as follows.

- *Encryption Parameters:* The BFV scheme chooses the parameters $\Theta = (n, p, q)$, where n is the polynomial modulus degree, p is the plaintext coefficient modulus of the polynomial ring $R_p = \mathbb{Z}_p[x]/\Phi_n(x)$, and q is the ciphertext coefficient modulus. It's worth noting that n is a positive power of 2, the polynomial modulus is the cyclotomic polynomial $\Phi_n(x) = x^n + 1$ with degree n, and q is chosen to be larger than p.
- *Encoding Γ_Θ and Decoding Γ_Θ^{-1}:* In this process, the raw message m is encoded as a polynomial $\tilde{m}$ by using binary representation [74], where the coefficients of the polynomial are $c_i = \{0, 1\}_{i=0,\ldots,n-1}$. For example, we can change 5 to a binary 0101, then encode 0101 as a polynomial $x^2 + 1$. Moreover, the decoding operation is an inverse operation of encoding, we omit the introduction of decoding here.
- *Keys Generation:* This stage generates the secret key msk and the public key mpk according to (1), where $c_{msk,i}$ takes values from the set $\{-1, 0, 1\}$, a is an n-degree polynomial with coefficients $c_{a,i}$ chosen from the set $\{0, \ldots, q-1\}$, and e is also an n-degree polynomial with coefficients randomly selected from a discrete and bounded Gaussian distribution [75].

$$msk = c_{msk,n-1}x^{n-1} + \cdots + c_{msk,1}x + c_{msk,0},$$
$$mpk = (mpk_0, mpk_1) = ([-(amsk + e)_{\Phi_n,q}, a]). \quad (1)$$

- *Encryption and Decryption:* The BFV scheme encrypts $\tilde{m}$ as $\tilde{m}'$ according to (2), where e_1 and e_2 are n-degree polynomials whose coefficients are similar to those of e, u is an n-degree polynomial whose coefficients are similar to those of msk, and $\lfloor \cdots \rfloor$ denotes the floor operation.

$$\tilde{m}' = (\tilde{m}'_0, \tilde{m}'_1) = \left(\left[mpk_0 u + e_1 + \left\lfloor \frac{q}{p} \right\rfloor \cdot \tilde{m} \right]_{\Phi_{n,q}}, [mpk_1 u + e_2]_{\Phi_{n,q}} \right). \quad (2)$$

The decryption process uses msk to decrypt $\tilde{m}'$ based on (3), where $\lfloor \cdots \rceil$ is round operation.

$$\tilde{m} = \left[\left\lfloor \frac{p}{q} [\tilde{m}'_0 + \tilde{m}'_1(msk)]_{\Phi_{n,q}} \right\rceil \right]_p. \quad (3)$$

- *Homomorphic Operation:* Given two encrypted messages $\tilde{m}'_1, \tilde{m}'_2$, the homomorphic addition is as (4),

$$\tilde{m}'_1 + \tilde{m}'_2 = ([\tilde{m}'_{10} + \tilde{m}'_{20}]_{\Phi_{n,q}}, [\tilde{m}'_{11} + \tilde{m}'_{21}]_{\Phi_{n,q}}), \quad (4)$$

and the homomorphic multiplication is as follows

$$\tilde{m}'_1 * \tilde{m}'_2 = ([\lfloor \tfrac{p}{q} \tilde{m}'_{10} \tilde{m}'_{20} \rceil]_{\Phi_{n,q}}, [\lfloor \tfrac{p}{q} (\tilde{m}'_{10} \tilde{m}'_{21} + \tilde{m}'_{11} \tilde{m}'_{20}) \rceil]_{\Phi_{n,q}}, [\lfloor \tfrac{p}{q} \tilde{m}'_{11} \tilde{m}'_{21} \rceil]_{\Phi_{n,q}}).$$

Notice that the homomorphic multiplication produces a three-term ciphertext, it can adopt relinearization technology [16] to reduce the size of the cipheretxt.

- *Noise Budget (NB):* NB is used to measure a maximum number of homomorphic operations while ensuring the correctness of the returned results. It is noteworthy that the NB decreases to 0 and results in incorrect ciphertext decryption.

B. SSE

The initial SSE scheme was introduced by Song et al. [76], and it is formally defined as follows:

- $KeyGen(1^\lambda)$: The key generation algorithm accepts the security parameter k as input, and outputs a secret key K.
- $Setup(K, DB)$: Using K and a plaintext database DB, this algorithm creates a searchable encryption index $\mathcal{I}$. The

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
GENERAL NOTATIONS IN THIS PAPER

NOTATIONS	MEANING
m	Original message
$\tilde{m}'$	The ciphertext of m
mpk	The pubic key of BFV
msk	The private key of BFV
HE	The homomorphic encryption algorithm of BFV
DE	The homomorphic decryption algorithm of BFV
$\mathbb{G}$	The multiplication group
f, g	The order and generator of $\mathbb{G}$
$ELUT$	The encryption look-up table
sk'	The secret key for constructing $ELUT$
$\tilde{\mathbf{X}}$	The ciphertext of each layer
$\mathbf{X}$	The plaintext of each layer

database DB is structured as $(id_i, W_i)i = 1^d$, where id_i represents the identifier of the i-th document, W_i denotes the keyword sets including the i-th document, and d is the total number of documents in DB.

- $Trapw(K, w)$: With K and a searchable keyword w, this algorithm generates a search trapdoor $Trap_w$.
- $Search(\mathcal{I}, Trap_w)$: The search algorithm takes $\mathcal{I}$ and $Trap_w$ as inputs and returns the results $DB(w)$, which include all documents containing w.

C. Deep Neural Network

The deep neural network is a framework of deep learning that typically includes linear modules and non-linear modules. Here, we'll provide a brief description as follows.

- *Linear Modules:* The linear modules include convolutional layer, average pooling layer and fully connected layer, where the convolutional layer performs a convolutional operation between the learned convolutional kernel and the image, the average pooling layer runs a down-sampling operation while maintaining the critical features of the image, and the fully connected layer takes the local features from previous layers and maps them into final predictions via a weight matrix, serving as the decision-making part of the network.
- *Non-Linear Modules:* The non-linear modules refer to activation functions such as ReLU, Tanh, Sigmoid, and Softplus. Among them, ReLU has gained widespread popularity due to its numerous advantages, including the absence of the gradient vanishing problem and the model's enhanced expressive capacity.

IV. CRYPTDNN ARCHITECTURE

In this section, we introduce the architecture, the threat model, and the design goals of CryptDNN. The meaning of symbols in this paper is shown in Table II.

A. Architecture

As shown in Fig. 1, the architecture of CryptDNN includes four parties: 1) Client; 2) Computation Platform (CP) and 3) Storage Platform (SP). The detailed description of the parties is as follows.

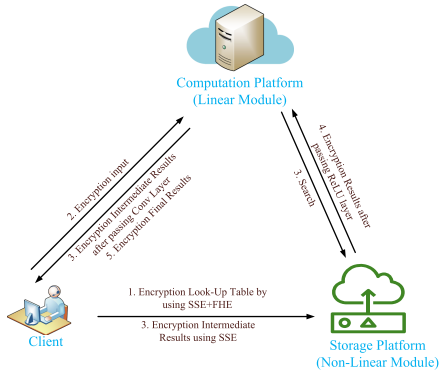


Fig. 1. The architecture of privacy-preserving inference on DNN.

1) *Client:* The client acts as a data provider responsible for storing his/her data and providing it to the DNN model to obtain the desired results. Therefore, the client mainly performs 4 functions: 1) generates the secret key pairs (msk, mpk) and sk' for the BFV scheme and the SSE scheme, respectively; 2) constructs an encryption look-up table $ELUT$ for the non-linear layer of the DNN model and sends it to the SP; 3) encrypts his/her input and the parameters of the DNN model by using mpk and sends it to the CP; 4) decrypts the intermediate results after passing the convolutional layer from the CP, and encrypts it by using sk' and sends it to the SP;
2) *CP:* The CP stores the DNN model and takes on the amount of computation operations of the DNN model. As a result, the CP mainly runs 3 operations: 1) computes the results in the convolutional layer and sends them to the client; 2) launches a search operation for the activation layer to the SP; 3) performs the computation operations of the remainder layers and sends the encryption results to the client.
3) *SP:* The SP successfully registered with the client, doubling as an edge server, operates as both a data storage and data search platform, it mainly performs 2 functions: 1) stores $ELUT$ from the client; 2) performs search operation on $ELUT$ and returns the results to the CP.

B. Threat Model and Design Goals

In CryptDNN, the client is assumed to be a trusted party, while the CP and the SP are considered honest-but-curious parties and the client does not collude with the CP and the SP. In other words, the client does not voluntarily disclose information, whereas the CP and the SP strictly adhere to the CryptDNN protocol, but they may opportunistically collect and acquire private information during inference process. For example, the CP and the SP will collect and acquire the original client data, the classified results, the model parameters and the intermediate data.

According to the above-mentioned threat model, the design goals of the CryptDNN are as follows.

- *Data privacy and model privacy:* The CryptDNN should guarantee the confidentiality both of the data (the client's

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Construction of $ELUT$.

Input: The pixel matrices $\mathbf{M}$, $HE, mpk, sk^{'}$ and PRF and the hash function H.
Output: The encryption look-up table $ELUT$.

Initialize an empty $ELUT \leftarrow \{\}$;
for i *in range* $(0, len(\mathbf{M})^2 - 1)$ **do**
  Compute the height $H_{\mathbf{M}_i}$ and the width $W_{\mathbf{M}_i}$ of $\mathbf{M}_i$;
  for h *in range*$(0, H_{\mathbf{M}_i} - 1)$ **do**
    for w *in range*$(0, W_{\mathbf{M}_i} - 1)$ **do**
      Encrypt the index $\hat{\mathbf{M}}_i[h][w]$ and the value $\tilde{\mathbf{M}}_i[h][w]$ of $ELUT$;
      $\hat{\mathbf{M}}_i[h][w] \leftarrow PRF_p(sk^{'}, H(i||h||w))$;
      if $\mathbf{M}_i[h][w] > 0$ **then**
        $\tilde{\mathbf{M}}_i[h][w] \leftarrow HE(mpk, \mathbf{M}_i[h][w])$;
      end
      else
        $\tilde{\mathbf{M}}_i[h][w] \leftarrow HE(mpk, 0)$;
      end
      Insert the key and value into $ELUT$;
      $ELUT.update(\{\hat{\mathbf{M}}_i[h][w] : \tilde{\mathbf{M}}_i[h][w]\})$;
    end
  end
end
Return $ELUT$.

Algorithm 2: Secure Convolutional Layer Computation.

Input: The encrypted pixel matrices $\tilde{\mathbf{M}}$, the encoded convolutional kernel $\tilde{\mathbf{W}}_k$, the encoded bias $\tilde{\mathbf{b}}_k$, the stride $\mathbf{s}$ and the padding pa.
Output: The convolutional results $\tilde{\mathbf{Z}}$.

if pa *is not None* **then**
  Choose $y_{pa} = pa[0], x_{pa} = pa[1]$;
  $zero = \tilde{\mathbf{M}}[:, y_{pa}+1, x_{pa+1}] - \tilde{\mathbf{M}}[:, y_{pa}+1, x_{pa+1}]$;
  Pad $\tilde{\mathbf{M}}$ by using $zero$;
end
for i *in range*$(0, len(\tilde{\mathbf{M}}) - 1)$ **do**
  Compute the dimension (H_{in}, W_{in}) of $\tilde{\mathbf{M}}_i$;
  for m *in range*$(0, k-1)$ **do**
    Compute the dimension (K_h, K_w) of $\tilde{\mathbf{W}}_m$;
    Extract a sub-matrix $\tilde{\mathbf{M}}_i[K_h, K_w]$[3] from $\tilde{\mathbf{M}}_i$;
    Compute the encryption convolutional operation $\tilde{\mathbf{C}}_m^i[K_h, K_w] = \tilde{\mathbf{M}}_i[K_h, K_w] \circ \tilde{\mathbf{W}}_m[K_h, K_w]$;
    Repeat the steps 10-12 based on $\mathbf{s}$ to obtain the finally pixel matrices $\tilde{\mathbf{Z}}_m^i[H_{out}, W_{out}]$ until the index of $\tilde{\mathbf{M}}_i[K_h, K_w]$ out the scope $[H_{in}, W_{in}]$;
    $H_{out} = \frac{(H_{in} - K_h + 2pa)}{\mathbf{s}[0]} + 1$;
    $W_{out} = \frac{(W_{in} - K_w + 2pa)}{\mathbf{s}[1]} + 1$;
    Add the bias $\mathbf{b}_m$ to $\tilde{\mathbf{Z}}_m^i$;
  end
end
for m *in range*$(0, k-1)$ **do**
  $\tilde{\mathbf{Z}}_m = \tilde{\mathbf{Z}}_m^1 + \tilde{\mathbf{Z}}_m^2 + \cdots + \tilde{\mathbf{Z}}_m^{len(\tilde{\mathbf{M}})}$;
end
$\tilde{\mathbf{Z}} = \{\tilde{\mathbf{Z}}_1, \cdots, \tilde{\mathbf{Z}}_m, \cdots, \tilde{\mathbf{Z}}_k\}$;
Return $\tilde{\mathbf{Z}}$.

data, the inference results, intermediate results) and the model parameter.

- *Inference accuracy:* The selection of the appropriate activation function is crucial for achieving high inference accuracy while maintaining privacy. Generally, the ReLU is considered the best activation function for achieving higher inference accuracy. Therefore, it is important to ensure that the ReLU function satisfies the privacy-preserving scheme.
- *Inference efficiency:* The CryptDNN leverages BFV homomorphic encryption technology to enable image inference. However, since the time costs associated with FHE are higher, the CryptDNN is designed with a focus on reducing the time costs of encrypted image inference.

V. Construction of CryptDNN

In this section, we present the detailed construction of the CryptDNN, including 6 parts: 1) initialization, 2) construction of $ELUT$, 3) secure convolutional layer computation, 4) secure activation layer search, 5) secure average pooling layer computation 6) secure fully connected layer computation.

A. Initialization

Given a security parameter 1^{λ}, the client first generates the public parameters $(f, g, \mathbb{G})$ and chooses random numbers $sk' = \alpha \in \mathbb{Z}_f^*$ as the secret key for constructing $ELUT$. Moreover, the client follows a "trial-and-error" approach to choose the BFV parameters $\Theta = (n, p, q)$, and then generates the private key $msk = k_s$ and the public key

$$mpk = (k_{p0}, k_{p1}) = ([-(ak_s + e)]_{\Phi_{n,q}}, a),$$

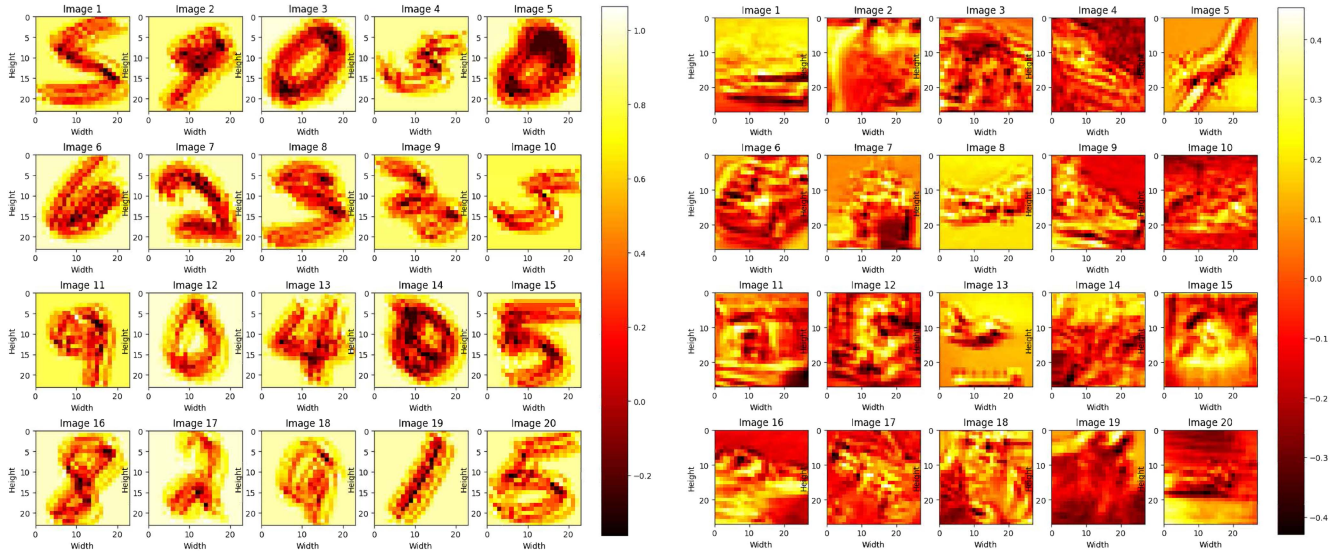
where k_s, a, e are three n-degree polynomial, the coefficients of k_s are randomly selected under the condition $k_s(x) \in \{-1, 0, 1\}$, the coefficients of a are randomly selected under the condition $a(x) \in \{0, 1, \ldots, q-1\}$, and the coefficients of e are randomly selected under the discrete and bounded Gaussian distribution $\mathcal{N}(\mu = 0, \sigma = 3.2)$.

B. Construction of $ELUT$

To explore the data distribution passed through the convolutional, we randomly choose 20 images in the MNIST dataset and the CIFAR10 dataset, then draw the heatmap of these images passed through the first convolutional layer. As shown in Fig. 2, we can learn the rough range of the pixels of these images passed through the first convolutional layer[4] is [-1,1] and most pixels in the images are consistent, if we use FHE to encrypt one by one, it costs much time and leads to the system infeasible.

[2] $len(\mathbf{M})$ denotes the number of matrices (channels) of $\mathbf{M}$.
[3] $[K_h, K_w]$ denotes the dimension of $\tilde{\mathbf{M}}_i$ is $K_h \times K_w$.
[4] We assume that the activation layer comes after the convolution layer in this paper.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.



(a) Heatmap of a MNIST image

(b) Heatmap of a CIFAR10 image (one channel)

Fig. 2. The heatmap of an image has passed through the first convolutional layer in the MNIST dataset and the CIFAR10 dataset.

To reduce the time cost of the non-linear layers' computation in the ciphertext model and continue to use the ReLU function as the activation function in the plaintext model, the client constructs an encryption look-up table $ELUT$ for the non-linear layers. In other words, when an encrypted image has passed the convolutional layer of the ciphertext model, it can search $ELUT$ to obtain the results of the non-linear layers rather than use the polynomial function to compute the results of the non-linear layers.

The construction process of $ELUT$ is shown in Algorithm 1. Specifically, the client uses normalization technology ($mean = [0.5], std = [0.5]$) to set the range of the pixels of each image be located in [-1,1]. Once the DNN model is well-trained, the client downloads the pixel matrices $\mathbf{M}$ which has passed through the convolutional layers, and then initializes an empty $ELUT$ which is structured like a hashmap. Furthermore, the client uses PRF and sk' to encrypt the elements in $\mathbf{M}$ as the index $\hat{\mathbf{M}}_i[h][w]$ of $ELUT$ following (5), where i is the number of $\mathbf{M}$ and h, w are the row and the column of $\mathbf{M}_i$, respectively.

$$\hat{\mathbf{M}}_i[h][w] = PRF_p(sk', H(i||h||w)). \tag{5}$$

Then the client employs the BFV scheme[5] to encrypt the elements in $\mathbf{M}$ as the value $\tilde{\mathbf{M}}_i[h][w]$ of $ELUT$ following (6). Finally, the client sends $ELUT$ to the SP for storage and performs a search operation by the SP, where the structure of $ELUT$ is shown in (7).

$$\tilde{\mathbf{M}}_i[h][w] = HE(mpk, \mathbf{M}_i[h][w]). \tag{6}$$

$$ELUT = \{\hat{\mathbf{M}}_1[h_{1,j}][w_{1,j}] : \tilde{\mathbf{M}}_1[h_{1,j}][w_{1,j}] \cdots \hat{\mathbf{M}}_i[h_{i,j}][w_{i,j}] : \tilde{\mathbf{M}}_i[h_{i,j}][w_{i,j}]\}_{j=len(\mathbf{M}_i)}^{i=len(\mathbf{M})} \tag{7}$$

[5]In fact, Pyfhel 2.x homomorphic encryption library lets the user use a fractional encoder, which makes it possible to use the BFV with fractional numbers with fixed precision.

C. Secure Convolutional Layer Computation

Given the parameters of the pre-trained model including the convolutional kernel $\mathbf{W}_k$ and the bias $\mathbf{b}_k$, where k is the number of convolutional kernel. The client uses the encoding method of the BFV to encode these parameters as $\tilde{\mathbf{W}}_k$ and $\tilde{\mathbf{b}}_k$ and then sends them to the CP. To obtain fine-grained features of the image to improve the inference accuracy of the model, the CP typically pads the elements by adding pa rows and pa columns around $\tilde{\mathbf{M}}$, where the padding elements are replaced by the encrypted zeros. The secure convolutional layer is computed as shown in Algorithm 2. Specifically, given the encrypted pixel matrices $\tilde{\mathbf{M}}$ and the encoded kernel matrices $\tilde{\mathbf{W}}_k$ and the encoded bias $\tilde{\mathbf{b}}_k$, the CP loop extracts a sub-matrix $\tilde{\mathbf{M}}_i[K_h, K_w]$ of $\tilde{\mathbf{M}}_i \in \tilde{\mathbf{M}}$, where i denotes the i-th pixel matrix of $\tilde{\mathbf{M}}$ and the height and the width of the sub-matrix are equal to the dimension (K_h, K_w) of $\tilde{\mathbf{W}}_m$, where $m \in [1, k]$.

Subsequently, the CP computes the encrypted Hadamard product $\tilde{\mathbf{C}}_m^i[K_h, K_w]$ between all of $\tilde{\mathbf{M}}_i[K_h, K_w]$ and $\tilde{\mathbf{W}}_m[K_h, K_w]$ according to (8), until there is no sub-matrix which can extract from $\tilde{\mathbf{M}}_i$, and then reshapes these Hadamard products to the final encrypted pixel matrices $\tilde{\mathbf{Z}}_m^i[H_{out}, W_{out}]$, where (H_{out}, W_{out}) denotes the dimension of the final encrypted pixel matrices have passed through encrypted convolutional operation, the computation formula of (H_{out}, W_{out}) is shown in (9), $\mathbf{s}$ is a list of stride. Then the CP adds the bias $\mathbf{b}_m$ to $\tilde{\mathbf{Z}}_m^i$.

$$\tilde{\mathbf{C}}_m^i[K_h, K_w] = \tilde{\mathbf{M}}_i[K_h, K_w] \circ \tilde{\mathbf{W}}_m[K_h, K_w]. \tag{8}$$

$$H_{out} = \frac{H_{in} - K_h + 2pa}{\mathbf{s}[0]} + 1,$$

$$W_{out} = \frac{W_{in} - K_w + 2pa}{\mathbf{s}[1]} + 1. \tag{9}$$

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

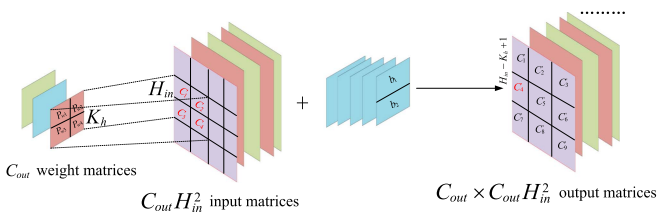


Fig. 3. Encrypted data processing flow under the convolutional layer.

Finally, the CP computes the final results $\tilde{\mathbf{Z}}$ based on (10) and sends a search signal, $\tilde{\mathbf{Z}}$ to the SP and the client, respectively. The visualization of convolutional operation is shown in Fig. 3.

$$\tilde{\mathbf{Z}}_m = \tilde{\mathbf{Z}}_m^1 + \tilde{\mathbf{Z}}_m^2 + \cdots + \tilde{\mathbf{Z}}_m^{len(\tilde{\mathbf{M}})},$$
$$\tilde{\mathbf{Z}} = \{\tilde{\mathbf{Z}}_1, \ldots, \tilde{\mathbf{Z}}_m, \ldots, \tilde{\mathbf{Z}}_k\}. \tag{10}$$

D. Secure Activation Layer Search

Traditional privacy-preserving DNN schemes using FHE typically employ low-degree polynomials or Taylor expansion at fixed points to substitute the activation function for performing homomorphic multiplication and addition under ciphertexts. However, these methods often compromise accuracy and efficiency because low-degree polynomials and Taylor expansions inherently lose some data information, and encrypting each data using FHE incurs additional computational overhead.

To mitigate these drawbacks, CryptDNN employs the search $ELUT$ to replace the computational operation of the ReLU function in the plaintext model. The search process of the non-linear layer (activation layer) under ciphertexts, delineated in Algorithm 3, involves 7 phases as follows.

- The client first decrypts the encrypted query data $\tilde{\mathbf{Z}}$ using the secret key msk and the decryption algorithm DE to obtain the plaintext matrix $\mathbf{Z}$, i.e.,
$$\mathbf{Z} = DE(msk, \tilde{\mathbf{Z}}).$$
- Then, the client generates the trapdoor matrix $\tilde{\mathbf{Z}}'$ by computing the pseudorandom tokens over each plaintext entry. Specifically, for each position (m, h, w), the token is generated as:
$$\tilde{\mathbf{Z}}'_m[h][w] = PRF_p(sk', H(m||h||w)).$$
- The client sends the trapdoor matrix $\tilde{\mathbf{Z}}'$ to the SP, which performs the secure search over the $ELUT$.
- For each location (m, h, w), the SP compares the trapdoor token $\tilde{\mathbf{Z}}'_m[h][w]$ with the index $\hat{\mathbf{M}}_i[h][w]$ stored in the $ELUT$. If they match, i.e.,
$$\tilde{\mathbf{Z}}'_m[h][w] = \hat{\mathbf{M}}_i[h][w], \tag{11}$$
the SP retrieves the corresponding encrypted value directly from the $ELUT$:
$$\tilde{\mathbf{SR}}_m[h][w] = ELUT.get(\hat{\mathbf{M}}_i[h][w]). \tag{12}$$
- Otherwise, the SP cannot find a match and returns the token $\tilde{\mathbf{Z}}'_m[h][w]$ to the client. The client then derives the original query value by computing the inverse hash position (i.e., using knowledge of (m, h, w)), and re-encrypts it using mpk:
$$\tilde{\mathbf{Z}}''_m[h][w] = HE(mpk, \mathbf{Z}_m[h][w]). \tag{13}$$
- The client sends the encrypted result $\tilde{\mathbf{Z}}''_m[h][w]$ back to the SP, who records it as the final search result:
$$\tilde{\mathbf{SR}}_m[h][w] = \tilde{\mathbf{Z}}''_m[h][w]. \tag{14}$$
- After completing all positions, the SP aggregates the result set:
$$\tilde{\mathbf{SR}} = \{\tilde{\mathbf{SR}}_1, \ldots, \tilde{\mathbf{SR}}_m\}$$
and returns it to the CP.

Algorithm 3: Secure Activation Layer Search.

Input: The encrypted data $\tilde{\mathbf{Z}}$ and the parameters $HE, DE, msk, mpk, ELUT, sk'$.
Output: The search results $\tilde{\mathbf{SR}}$.

```
The client decrypts Z̃ into Z = DE(msk, Z̃);
The client generates the trapdoor Z̃' = PRF_p(sk', Z);
The client sends Z̃' to the SP;
The SP searches ELUT;
The SP computes the channels and the dimension of Z̃' as (Z_out, Z_h, Z_w);
for i, m in range(0, Z_out − 1) do
    The SP initializes an empty matrix SR̃_i;
    for h in range(0, Z_h − 1) do
        for w in range(0, Z_w − 1) do
            if Z̃'_m[h][w] == M̂_i[h][w] then
                The SP records SR̃_m[h][w] = ELUT.get(M̂_i[h][w]);
            end
            else
                The SP sends Z̃'_m[h][w] to the client;
                The client decrypts Z̃'_m[h][w] into Z'_m[h][w] = PRF_p(sk', Z̃'_m[h][w]);
                The client encrypts Z'_m[h][w] into Z̃''_m[h][w] = HE(mpk, Z'_m[h][w]);
                The client sends Z̃''_m[h][w] to the SP;
                The SP records SR̃_m[h][w] = Z̃''_m[h][w]
            end
        end
    end
end
SR = {SR_1, ··· , SR̃_m};
Return SR to the CP.
```

E. Secure Average Pooling Layer Computation

Most of the existing DNN adopts maximum pooling as the pooling layer, but it is not compatible with FHE as the same as the ReLU function because the FHE cannot compare the biggest value between numbers in the ciphertext scenarios. To solve this problem, the CryptDNN utilizes an average pooling layer

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

to replace the max pooling layer. The secure average pooling computation is shown in Algorithm 4.

Specifically, given a m-th encrypted feature image $\widetilde{\mathbf{SR}}_m$, the CP first pads $\widetilde{\mathbf{SR}}_m$ by using padding factor pa to obtain more features in the pooling computation process. Subsequently, the CP computes the input channel C_{in}, the height H_{in} and the width W_{in} of $\widetilde{\mathbf{SR}}_m$, and then computes the height k_h and the width k_w of $\widetilde{\mathbf{W}}$. Since the average pooling layer does not increase the number of output feature images, it reduces the dimension of output feature images, thus the output channel is equal input channel, i.e., $C_{out} = C_{in}$.

To obtain the encrypted average pooling results $\widetilde{\mathbf{AP}}_m$, the CP computes the output dimension $(C_{out}, H_{\widetilde{\mathbf{AP}}_m}, W_{\widetilde{\mathbf{AP}}_m})$ of $\widetilde{\mathbf{AP}}_m$ according to (15). In addition, the CP extracts a sub-matrix $\widetilde{\mathbf{SR}}_m[i{+}\mathbf{s}[0]:i{+}\mathbf{s}[0]{*}k_h, j{+}\mathbf{s}[1]:j{+}\mathbf{s}[1]{*}k_w]$ from $\widetilde{\mathbf{SR}}_m$ and then computes its mean value based on (16) as the element of the i-th row and the j-th column of $\widetilde{\mathbf{AP}}_m$, where $i \in [0, H_{\widetilde{\mathbf{AP}}_m} - 1], j \in [0, W_{\widetilde{\mathbf{AP}}_m} - 1]$.

$$H_{\widetilde{\mathbf{AP}}_m} = \frac{(H_{in} - k_h + 2pq)}{\mathbf{S}[0]} + 1,$$
$$W_{\widetilde{\mathbf{AP}}_m} = \frac{(W_{in} - k_w + 2pq)}{\mathbf{S}[1]} + 1. \tag{15}$$

$$\widetilde{\mathbf{AP}}_m[i][j] = \frac{\sum \widetilde{\mathbf{SR}}_m[i{+}\mathbf{s}[0]:i{+}\mathbf{s}[0]{*}k_h, j{+}\mathbf{s}[1]:j{+}\mathbf{s}[1]{*}k_h]}{\widetilde{sf}}. \tag{16}$$

F. Secure Fully Connected Layer Computation

In this phase, the CP first encodes the kernel $\mathbf{K}'$ and the bias $\mathbf{b}'$ into $\widetilde{\mathbf{K}}'$ and $\widetilde{\mathbf{b}}'$, respectively. Subsequently, the CP flattens $\widetilde{\mathbf{AP}}$ into a vector $\widetilde{\mathbf{ap}}$ with the dimension of $m \times H_{\widetilde{\mathbf{AP}}_m} \times W_{\widetilde{\mathbf{AP}}_m}$, and then serves it as the input of the fully connected layer. Besides, the CP computes the final results $\mathbf{y}$ as the output of the fully connected layer using the following (17), where $n = m \times H_{\widetilde{\mathbf{AP}}_m} \times W_{\widetilde{\mathbf{AP}}_m}$, k is the number of the output features. Finally, the CP sends these results to the client. The client then decrypts $\mathbf{y}$ using the secret key msk and reads the decrypted output. This process ensures the confidentiality of the computation results while enabling the client to access the final outcomes.

$$\begin{aligned}\mathbf{y} &= (y_1, y_2, \ldots, y_k) = \widetilde{\mathbf{ap}}\widetilde{\mathbf{W}}'^{T} + \widetilde{\mathbf{b}}' \\ &= (x_1, \cdots x_n)\begin{pmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \vdots & \vdots \\ w_{k1} & \cdots & w_{kn} \end{pmatrix}^{T} + (b_1, \ldots, b_k). \end{aligned} \tag{17}$$

VI. Security Analysis

In this section, our main focus is on analyzing how CryptDNN safeguards data privacy, model privacy, and index privacy.

A. Data Privacy and Model Privacy

In CryptDNN, the client is assumed to be an honest party, and the CP and the SP are assumed to be two honest-but-curious parties. The client initially generates the keys (msk, mpk) of the scheme and broadcasts mpk to the CP. Then the client uses mpk to encrypt their data and uploads it to the CP. Recalling Section III-A, we know the public key mpk and the encryption step as follows

$$mpk = \left([-(a(msk) + e)]_{\Phi_{n,q}}, a\right),$$
$$\tilde{m}' = ([(-a(msk){+}e)\mu{+}e_1 + \left\lfloor \frac{q}{p} \right\rfloor \cdot m]_{\Phi_{n,q}}, [a\mu + e_2]_{\Phi_{n,q}}).$$

Since μ, e_1, and e_2 are random polynomials, an interesting phenomenon arises where the same message after applying the BFV scheme yields different ciphertexts, thereby satisfying the probabilistic encryption attributes of FHE. It is worth noting that the private key msk is solely retained by the client, ensuring that the CP cannot decrypt the ciphertext without access to the private key. This setup significantly enhances data privacy protection. Likewise, the client employs the same method to safeguard the parameters of the DNN model and then sends these encrypted parameters to the CP, thereby ensuring the confidentiality of the model in the CP. Furthermore, the security of the BFV scheme hinges on the RLWE problem. Readers interested in a comprehensive security proof are encouraged to refer to [77].

Algorithm 4: Secure Average Pooling Computation.

Input: The encrypted data $\widetilde{\mathbf{SR}}$, the encoded pooling kernel $\widetilde{\mathbf{W}}$, the stride $\mathbf{s}$, the padding pa.
Output: The average pooling results $\widetilde{\mathbf{AP}}$.

```
if pa is not None then
    Choose y_pa = pa[0], x_pa = pa[1];
    zero = SR[y_pa + 1][x_pa+1] − SR[y_pa + 1][x_pa+1];
    Pad SR by using zero;
end
for m in range(0, C_in − 1) do
    Compute (C_in, H_in, W_in) ← SR_m;
    Compute (C_out, k_h, k_w) ← W_m;
    Compute the scale factor sf = 1/(k_h * k_w);
    Encode the scale factor sf ← sf;
    Compute the height H_AP of AP
      H_AP = ((H_in − k_h + 2pq)/S[0]) + 1;
    Compute the width W_AP of AP
      W_AP = ((W_in − k_w + 2pq)/S[1]) + 1;
    for i in range(0, H_AP − 1) do
        for j in range(0, H_AP − 1) do
            Extract a sub-matrix
              SR_m[i+s[0]:i+s[0]*k_h, j+s[1]:j+s[1]*k_w];
            Compute the encrypted average pooling
              operation AP_m[i][j] =
              (Σ SR_m[i+s[0]:i+s[0]*k_h, j+s[1]:j+s[1]*k_h]) / sf;
        end
    end
end
Return AP.
```

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

B. Index Privacy

Index privacy means the client constructs an encrypted look-up table and uploads it to the storage platform in a manner that prevents the storage platform from obtaining any information contained within the encrypted look-up table.

Theorem 1: The $ELUT$ of CryptDNN is index indistinguishability under the same-closeness-pattern chosen-plaintext attacks (IND-CLS-CCA).

Proof: We assume there are two parties, one is an adversary $\mathcal{A}$ and the other is a challenger $\mathcal{B}$, then we pass through the interactive game between $\mathcal{A}$ and $\mathcal{B}$ to prove the index security of the CryptDNN.

- *Initialization: The CryptDNN prepares two look-up tables LUT_1, LUT_2 with the same number of elements, then $\mathcal{A}$ sends LUT_1, LUT_2 to $\mathcal{B}$.*
- *Setup: $\mathcal{B}$ utilizes the encryption key sk' and mpk to encrypt the key and value of the look-up table, respectively. Note that in the look-up table, the key and value represent the same elements, denoting the output of an image after passing through the convolutional layer.*
- *Phase 1: $\mathcal{A}$ uploads some queries to $\mathcal{B}$, where the queries include 2 types.*
  1) *Index generation: $\mathcal{A}$ selects a look-up table $LUT_i' = \{k_{1i} : v_{1i}, k_{2i} : v_{2i}, \ldots, k_{ni} : v_{ni}\}$ as the i-th output after passing through the convolutional layer, and sends them to $\mathcal{B}$. Once $\mathcal{B}$ receives LUT_i', it immediately employs sk' and mpk to encrypt the key and value of LUT', and outputs the encrypted look-up table $ELUT'$.*
  2) Query: *$\mathcal{A}$ chooses a query matrix $\mathbf{Q}$to $\mathcal{B}$, then $\mathcal{B}$ uses sk' to encrypt their $\mathbf{Q}'$.*
- *Challenge: After receiving LUT_1, LUT_2, $\mathcal{B}$ selects a random value $b \in \{0, 1\}$ and employs sk' and mpk to encrypt LUT_1, LUT_2, then outputs the encryption look-up table $ELUT_b$.*
- *Phase 2: $\mathcal{A}$ first duplicates the phase 1, and then sends the corresponding results to $\mathcal{B}$.*
- *Guess: $\mathcal{A}$ outputs b'.In summary, $\mathcal{A}$ faces significant difficulty in recovering the plaintext context from the encryption look-up table $ELUT$ without access to the keys sk' and msk. In other words, $\mathcal{A}$ cannot distinguish between the encryption look-up tables $ELUT_1$ and $ELUT_2$. In addition, we can compute the probability $Pr_{\mathcal{A}}$ under the condition $b = b'$ as:*

$$Pr_{\mathcal{A}} = \left| Pr(b' = b) - \frac{1}{2} \right| = \left| \left(\frac{1}{2} + \frac{1}{n} \right) - \frac{1}{2} \right| = \frac{1}{n},$$

where n denotes the number of guesses made by $\mathcal{A}$. Thus, the probability of $\mathcal{A}$ winning the game is negligible. Therefore, CryptDNN achieves index security.

■

Recalling the construction process of $ELUT$, we know $ELUT$ is a key-value structure like a hash table, if the CP colludes with the SP, they do not know any information of $ELUT$ without sk' and msk. Therefore, the CryptDNN can safeguard the confidentiality of both the data and the model.

TABLE III
MUTIPLICATION DEPTH COMPARISON

Schemes	*Datasets*	*Model Depth*	*Original*	*Encryption*
CryptDNN	**MNIST**	**8**	**5**	**2**
	CIFAR10	**9**	**6**	**2**
CryptoNets [13]	MNIST	8	5	5
	CIFAR10	9	6	6
BgvDNN [15]	MNIST	8	5	5
	CIFAR10	9	6	6
BfvDNN [21]	MNIST	8	5	5
	CIFAR10	9	6	6
TfheDNN [26]	MNIST	8	5	5
	CIFAR10	9	6	6
CkksDNN [24]	MNIST	8	5	2
	CIFAR10	9	6	2

Original is the maximum number of multiplications of the original model; *Encryption* is the maximum number of multiplications of the encryption model.

VII. PERFORMANCE ANALYSIS

In this section, we mainly compare the inference accuracy and inference efficiency of the CryptDNN with other schemes, such as [13], [15], [21], [24], [26]. Furthermore, we also analyze the storage costs and time costs of constructing the encryption look-up table of the CryptDNN. We utilize two primary datasets for our experiments in CryptDNN: MNIST [78] and CIFAR10 [79]. MNIST datasets consist of 60,000 training handwritten digits and 10,000 validation digits ranging from "0" to "9". Each handwritten digit is represented by 784 features, corresponding to a 28×28 pixel image. On the other hand, CIFAR10 datasets comprise 50,000 training images and 10,000 test images, encompassing 10 categories of RGB color images, where each RGB image in CIFAR10 has dimension of 32×32 pixels. The code link is https://github.com/lilvmy/CryptDNNExperiment.

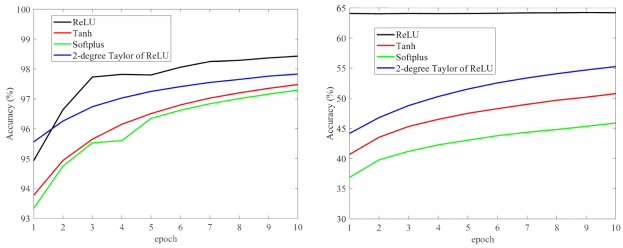
Furthermore, all experiments are executed in Intel core i7-12700 12-core CPU @2.1GHZ and 32 GB RAM running on the Linux operating system, and the BFV scheme is implemented by using Pyfhel [80] homomorphic encryption libraries. Moreover, the parameters of the BFV scheme are set $(n = 4096, p = 9539837271)$ and we use AES-256 as our PRF operation and employ SECP256R1 [81] to generate the secret key. For MNIST, the DNN model includes two convolutional layers (with 4 and 12 output channels), each followed by 2×2 average pooling, and ends with a fully connected layer that maps 192 features to 10 output classes. For CIFAR10, the model has two convolutional blocks (with 32 and 64 channels), each followed by ReLU activation and average pooling, and concludes with two fully connected layers that reduce the features from 576 to 128 and then to 10 classes. However, it's essential to note that the accuracy of these simplified models may not match that of established models like VGG-16 or AlexNet. We assume that the CryptDNN consumes one multiplication for every ciphertext multiplication performed. Since the CryptDNN conducts separate computation on the activation layer, the maximum number of multiplications required by the CryptDNN will be reset and recalculated within the activation layer. The number of multiplications required by the CryptDNN is compared with other homomorphic encryption network models under our constructed DNN model, as depicted in Table III. From Table III, it's notable that CryptDNN only requires 2 multiplications to complete a secure inference, whereas

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF COMPUTATIONAL AND COMMUNICATIONAL COMPLEXITY

Scheme	Computational Complexity	Communicational Complexity
SecureML [46]	$O(t \times n_1 \times d \times \lambda)$	$O(t \times n_1 \times d \times \lambda^2)$
MiniONN [48]	$O(L \times n_2 \times \lambda^2)$	$O(L \times n_2 \times \lambda^2)$
SIRNN [51]	$O(m \times l \times \lambda^2)$	$O(m \times l \times \lambda^2)$
Chettah [50]	$O(L \times n_1 \times \lambda^2)$	$O(L \times n_1 \times \lambda^2)$
BfvDNN [21]	$O(L \times p^2 \times \lambda)$	–
TfheDNN [26]	$O(L \times n_2 \times s \times \lambda^2)$	–
CryptDNN	$O(((L-1)+\|M\|) \times p^2 \times \lambda)$	$O(\|M\| \times p \times \lambda)$

Notes: n_1 is the number of the data samples, t is the number of the iteration, d is the dimension of feature of the image, λ is the security parameter, n_2 is the number of the neurons, m is the number of math functions, l is the bits, p is the degree of the polynomial, s is the number of programmable self-guided, L is the number of layers, $|M|$ is the number of feature map elements after the image passes through the convolution layer and – denotes the scheme does not involve data transmission.



(a) MNIST (b) CIFAR10

Fig. 4. Accuracy of using different activation functions in different datasets.

other schemes remain consistent with the original model. Consequently, compared to other schemes, CryptDNN markedly reduces the maximum number of multiplications for ciphertext DNN.

A. Complexity Analysis

In this section, we focus on comparing the computational and communicational complexity of CryptDNN with other schemes which use different encryption methods, such as kinds of literature, SecureML [46], MiniONN [48], SIRNN [51], Chettah [50], BfvDNN [21] and TfheDNN [26]. The comparison results are shown in Table IV.

In SecureML, the computational complexity mainly involves the activation function and matrix multiplication, thus the computational complexity is $O(t \times n_1 \times d \times \lambda)$, and the communicational complexity consists of the data sharing, offline exchange, and the number of iteration, so the communicational complexity is $O(t \times n_1 \times d \times \lambda)$. In MiniONN, the computational complexity includes linear transformation, activation function and pooling operation, so the computational complexity is $O(L \times n_2 \times \lambda^2)$ while communicational complexity is $O(L \times n_2 \times \lambda^2)$ which consists of secret sharing and Oblivious transfer and offline pre-computing. In SIRNN, the computational complexity involves complex math function evaluation and layer evaluation, thus the computational complexity is $O(m \times l \times \lambda^2)$ and the communicational complexity is $O(m \times l \times \lambda^2)$. In Chettah, they primarily involve secure matrix multiplications, activation functions, data exchange and other layer-specific computations under encryption or multi-party computation protocols, so the computational and the communicational complexities are $O(L \times n_1 \times \lambda^2)$ and $O(L \times n_1 \times \lambda^2)$, respectively.

The schemes mentioned above are hybrid schemes, i.e., combining homomorphic encryption or secret sharing with secure multi-party computation, and then we mainly introduce the computational complexity which only uses FHE technology. In BfvDNN and TfheDNN, the computational complexity primarily involves polynomial multiplication, polynomial addition, and polynomial approximation, etc., so the computational complexity of BfvDNN is $O(L \times p^2 \times \lambda)$. Different from BfvDNN, nonlinear activation functions in TfheDNN are implemented using programmable bootstrapping and look-up tables, which require multiple polynomial operations per activation function computation, thus the computational complexity is $O(L \times n_2 \times s \times \lambda^2)$. Since these two schemes do not involve data transmission, thus we do not consider the communicational complexity.

CryptDNN is also a hybrid scheme that combines SSE and FHE, where SSE is used to build an encrypted look-up table for the activation layer and FHE uses the BFV homomorphic encryption which is like BfvDNN. Specifically, the computational complexity of building $ELUT$ involves the index generation and the data encryption, where the computation complexity of building the index, encrypting data and constructing hash table are $O(|M| \times \lambda)$, $O(|M| \times p^2 \times \lambda)$ and $O(|M|)$, respectively. Moreover, the computational complexity of searching $ELUT$ is $O(\log |M|)$. For the linear computation, the computational complexity is similar to the BfvDNN scheme, i.e., $O((L-1) \times p^2 \times \lambda)$. Since $O(|M| \times \lambda), O(|M|), O(\log |M|) \ll O(|M| \times p^2 \times \lambda)$, thus the computational complexity is $O(((L-1)+|M|) \times p^2 \times \lambda)$. Furthermore, the communication overhead arises only at the non-linear activation layers, where interaction is required among the client, the CP, and the SP. For each activation layer, the communication process includes: (1) the CP transmits $|M|$ BFV-encrypted intermediate activations to the client, thus the communicational complexity is $\mathcal{O}(|M| \times p \times \lambda)$, (2) the client decrypts and re-encrypts these using a PRF-based SSE scheme and sends them to the SP for secure look-up, thus the communicational complexity is $O(|M| \times \lambda)$, (3) the SP returns the encrypted activation outputs to the CP, thus the communicational complexity is $\mathcal{O}(|M| \times p \times \lambda)$. Since each ciphertext has bit-length $\mathcal{O}(p \times \lambda)$, and there are $|M|$ such activations per non-linear layer, the overall communication complexity per activation layer is $\mathcal{O}(|M| \times p \times \lambda)$.

In summary, the computational and communicational complexities of the CryptDNN are lower than other comparison schemes.

B. Performance Analysis on MNIST and CIFAR10

1) Inference Accuracy and Efficiency: To evaluate the inference accuracy with different activation functions, such as ReLU, Tanh, Softplus, and low-degree Taylor expansion of ReLU at fixed point zero, we have experimented with the inference accuracy with different activation functions of the MNIST datasets and CIFAR10 datasets under 10 epochs as shown in Fig. 4. Note that these two experiments demonstrate that CryptDNN can select the activation function whose inference accuracy is higher. Based on the observations from Fig. 4(a) and (b), it is evident that the inference accuracy achieved with ReLU surpasses that

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

TABLE V
THE INFERENCE ACCURACY AND EFFICIENCY WITH DIFFERENT FHE SCHEMES ON MNIST AND CIFAR10 DATASETS

Datasets	*Schemes*	*Inference Accuracy*	*Inference Efficiency*	*Time Costs of $ELUT$*
MNIST	Plain Model	98.97%	0.145s	NA
	CryptoNets [13]	97.79%	248.75s	NA
	BgvDNN [15]	97.95%	218.46s	NA
	BfvDNN [21]	98.26%	185.62s	NA
	TfheDNN [26]	97.63%	157.799s	NA
	CkksDNN [24]	98.92%	200.86s	NA
	CryptDNN	**98.94%**	**89.501s**	3.25s
CIFAR10	Plain Model	64.58%	0.145s	NA
	CryptoNets [13]	63.89%	1637.578s	NA
	BgvDNN [15]	63.96%	1884.695s	NA
	BfvDNN [21]	64.21%	1448.299s	NA
	TfheDNN [26]	63.47%	1408.374s	NA
	CkksDNN [24]	64.52%	2440.667s	NA
	CryptDNN	**64.54%**	**1221.583s**	12.25s

Note: For CIFAR10, CryptDNN uses a simple DNN instead of a complex model like VGG-16 to demonstrate feasibility.

of other activation functions. Consequently, employing ReLU in the ciphertext model could potentially yield higher inference accuracy compared to using low-degree polynomial approximations, which are typically employed to ensure compatibility with FHE.

Table V presents the inference accuracy and efficiency of the ciphertext model across various schemes under the MNIST and CIFAR10 datasets, along with the time costs associated with constructing the encryption look-up table ($ELUT$) in CryptDNN, where the entry NA denotes the scheme that does not involve the construction of an encrypted lookup table. Note that we test only an encrypted image to illustrate the inference efficiency and time cost of $ELUT$. From Table V, we can learn that the inference accuracy of CryptDNN and CkksDNN [24] is almost the same as the plain model and slightly higher than other comparison schemes. This phenomenon arises because schemes like CryptoNets [13], BfvDNN [21], and TfheDNN [26] employ low-dimensional polynomial approximations instead of ReLU as the activation function in the encryption model. While CkksDNN [24] resembles CryptDNN in splitting the DNN model and using ReLU as the activation function, it decrypts the intermediate data after the convolutional layer before applying the ReLU function, thereby disclosing the intermediate data.

In addition, the inference efficiency of MNIST datasets in Table V indicates that CryptDNN is 1.6-2.6 times faster than other comparison schemes and Table V indicates that CryptDNN reduces 100s-1100 s inference time costs of CIFAR10 datasets compared with other schemes. This efficiency is achieved because CryptDNN converts the activation function computation operation into a search operation, significantly reducing the inference time costs. The construction of $ELUT$ is carried out by the client and stored in the storage platform, where multi-threading can be utilized to diminish construction overheads, rendering them negligible for CryptDNN. The time costs of each layer in CryptDNN under the MNIST and CIFAR10 datasets are depicted in Table VI, it is evident that the convolutional layer incurs higher time costs compared to other layers. This is primarily because the convolutional layer involves computing the multi-channel sum of weights between the encrypted image and the convolutional kernel.

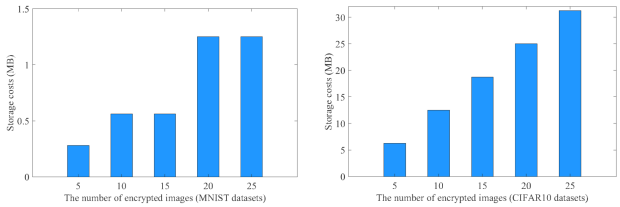
2) Communicational Overheads Comparison: We compare the communicational overheads between the alternative architectures and the CryptDNN, as shown in Table VII. In

TABLE VI
THE TIME COSTS OF EACH LAYER COMPUTATION UNDER MNIST AND CIFAR10 DATASETS IN CRYPTDNN

Datasets	*Layers*	*Construction Overheads*
MNIST	**Convolutional Layer1**	**25.383s**
	Activation Layer	0.780s
	Average Pooling Layer	0.255s
	Convolutional Layer2	**33.607s**
	Average Pooling Layer	0.087s
	Fully Connected Layer	7.967s
CIFAR10	**Convolutional Layer1**	**834.2002s**
	Activation Layer	4.5837s
	Average Pooling Layer	2.852s
	Convolutional Layer2	**295.5894s**
	Average Pooling Layer	0.265s
	Fully Connected Layer	33.636s

TABLE VII
THE COMMUNICATIONAL OVERHEADS WITH DIFFERENT ALTERNATIVE ARCHITECTURES ON MNIST AND CIFAR10 DATASETS

Scheme	*Datasets*	*Communicational Overhead*
MiniONN [48]	MNIST	657.5 MB
	CIFAR10	9272 MB
SIRNN [51]	MNIST	24 MB
	CIFAR10	360 MB
Chettah [50]	MNIST	7.5 MB
	CIFAR10	30 MB
CryptDNN (ours)	**MNIST**	**18.6 KB**
	CIFAR10	**196.6 KB**



(a) MNIST dataset. (b) CIFAR10 dataset.

Fig. 5. Storage overheads of constructing $ELUT$ with different datasets.

Table VII, CryptDNN incurs only kilobytes of communication per inference, whereas competing schemes remain in the megabyte range. This dramatic reduction stems from CryptDNN's use of a $ELUT$ for each non-linear layer: the $ELUT$ is created once during an offline setup phase and is never transmitted again. During the inference process, each ReLU is computed with a single oblivious table query that involves one ciphertext and a small index, while the remaining linear layers are processed with lightweight FHE operations. Consequently, the client, the CP and the SP communicate only a few times throughout the entire network evaluation. In contrast, MiniONN, SIRNN, and Cheetah execute an interactive sub-protocol after every layer, e.g., garbled circuits for activations in MiniONN and SIRNN, and Beaver-style multiplications with synchronization rounds in Cheetah, thus causing their communication cost to grow linearly with the depth of the DNN.

3) Storage Costs of $ELUT$: We evaluate the storage costs using the first 25 encrypted images from the MNIST and CIFAR10 datasets, and the storage overheads of the $ELUT$ are depicted in Fig. 5. It is apparent that the storage overheads exhibit a linear relationship with the number of encrypted images. However, the rate of increase in Fig. 5(a) is relatively slow due to the

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

presence of repeated values when an encrypted image has passed through the convolutional layer. It's important to highlight that the abundance of identical pixel values within images from the MNIST datasets which results in minimal changes in storage costs between 10 and 15 encrypted images, as well as between 20 and 25 encrypted images. Furthermore, the storage overheads for constructing $ELUT$ in the CIFAR10 datasets are notably higher compared to the MNIST datasets in Fig. 5(b). This discrepancy arises from the multi-channel structure presented in CIFAR10 datasets and the higher dimension of pixel matrices involved. For instance, after the initial convolutional layer, MNIST datasets have 3 channels, whereas CIFAR10 datasets have 32 channels. In addition, the dimensions of pixel matrices for MNIST datasets are 24×24, while those for CIFAR10 datasets are 28×28.

In summary, the CryptDNN not only achieves higher inference accuracy but also reduces inference time costs, making it more adaptable in real-world applications.

VIII. Conclusion

To solve the limitations of existing privacy-preserving DNN inference schemes with FHE in the IoT domain, such as the lower accuracy and efficiency, we have proposed a fast privacy-preserving DNN inference architecture based on cloud-edge-client collaboration, named as CryptDNN in this paper. Specifically, we have first used the BFV scheme to encrypt the data and encode the DNN model to guarantee the confidentiality of the data and the DNN model. Then we split the DNN model into linear modules and non-linear modules, where the linear modules and non-linear modules are encrypted and stored in the cloud server and the edge server, respectively. To ensure the privacy of the non-linear modules, we have utilized the proposed SSE technology to encrypt the data of non-linear modules, thus creating an encryption look-up table. This approach aims to tackle the accuracy and efficiency issues related to approximating the activation function using low-degree polynomials under ciphertext conditions. The final security and performance analyses demonstrate that CryptDNN can achieve higher inference accuracy and efficiency in the realistic MNIST datasets and CIFAR10 datasets compared to other works in the literature.

The limitation of the CryptDNN is that GPU is not exploited at all in the ciphertext scenario. Some recent works integrated ciphertext models into a GPU [82] which potentially resulted in a much faster secure DNN inference system compared to the existing ones, exploring how to use this method to further enhance cryptographic techniques will be an intriguing and challenging future research direction.

References

[1] J. Gao, Z. Wang, and X. Wei, "An adaptive pricing framework for real-time AI model service exchange," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 5114–5129, Sep.–Oct. 2024.

[2] J. Wang et al., "Generative AI based secure wireless sensing for ISAC networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 5195–5210, 2025.

[3] J. Ran and D. Li, "A faster privacy-preserving medical image diagnosis scheme with machine learning," *J. Imag. Inform. Med.*, pp. 1–13, 2025.

[4] D. Bala et al., "MonkeyNet: A robust deep convolutional neural network for monkeypox disease detection and classification," *Neural Netw.*, vol. 161, pp. 757–775, 2023.

[5] D. Shang, Z. Yan, L. Zhang, and Z. Cui, "Digital financial asset price fluctuation forecasting in digital economy era using blockchain information: A reconstructed dynamic-bound Levenberg–Marquardt neural-network approach," *Expert Syst. Appl.*, vol. 228, 2023, pp. 120329.

[6] L. Sun, X. Huang, and Y. Song, "Distributed event-triggered control of networked strict-feedback systems via intermittent state feedback," *IEEE Trans. Autom. Control*, vol. 68, no. 8, pp. 5142–5149, Aug. 2023.

[7] Z. Sun, G. Sun, Y. Liu, J. Wang, and D. Cao, "BARGAIN-MATCH: A game theoretical approach for resource allocation and task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1655–1673, Feb. 2024.

[8] J. Li, G. Sun, L. Duan, and Q. Wu, "Multi-objective optimization for UAV swarm-assisted IoT with virtual antenna arrays," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4890–4907, May 2024.

[9] L. Sun, X. Huang, and Y. Song, "Decentralized intermittent feedback adaptive control of non-triangular nonlinear time-varying systems," *IEEE Trans. Autom. Control*, vol. 69, no. 2, pp. 1265–1272, Feb. 2024.

[10] C. Gentry, S. Halevi, and V. Vaikuntanathan, "A simple BGN-type cryptosystem from LWE," in *Proc. 29th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2010, pp. 506–522.

[11] D. Li, J. Wu, J. Le, Q. Lü, X. Liao, and T. Xiang, "An efficient privacy-preserving ranked multi-keyword retrieval for multiple data owners in outsourced cloud," *IEEE Trans. Serv. Comput.*, vol. 17, no. 2, pp. 406–419, Mar.–Apr. 2024.

[12] D. Li, X. Liao, T. Xiang, J. Wu, and J. Le, "Privacy-preserving self-serviced medical diagnosis scheme based on secure multi-party computation," *Comput. Secur.*, vol. 90, 2020, Art. no. 101701.

[13] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.

[14] J. W. Bos, K. Lauter, J. Loftus, and M. Naehrig, "Improved security for a ring-based fully homomorphic encryption scheme," in *Proc. 14th IMA Int. Conf.*, 2013, pp. 45–64.

[15] H. Chabanne, A. De Wargny, J. Milgram, C. Morel, and E. Prouff, "Privacy-preserving classification on deep neural network," *Cryptol. ePrint Arch.*, pp. 1–18, 2017.

[16] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptol. ePrint Arch.*, pp. 1–19, 2012.

[17] Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) LWE," *SIAM J. Comput.*, vol. 43, no. 2, 2014.

[18] T. Debadrita, M. Nimish, B. Arnab, P. Sikhar, and M. Debdeep, "SEC: Fast private Boolean circuit evaluation from encrypted look-ups," *Cryptol. ePrint Arch.*, pp. 1–37, 2023.

[19] J. B. Truong, W. Gallagher, T. Guo, Tian, and R. J. Walls, "Memory-efficient deep learning inference in trusted execution environments," in *Proc. IEEE Int. Conf. Cloud Eng.*, 2021, pp. 161–167.

[20] S. Fei, Z. Yan, H. Xie, and G. Liu, "Sec-E2E: End-to-end communication security in LS-HetNets based on blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 761–778, Jan.–Feb. 2024.

[21] A. Falcetta and M. Roveri, "Privacy-preserving deep learning with homomorphic encryption: An introduction," *IEEE Comput. Intell. Mag.*, vol. 17, no. 3, pp. 14–25, Aug. 2022.

[22] R. Dathathri et al., "CHET: An optimizing compiler for fully-homomorphic neural-network inferencing," in *Proc. 40th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2019, pp. 142–156.

[23] W. Yang, X. Wang, Z. Guan, L. Wu, X. Du, and M. Guizani, "SecureSL: A privacy-preserving vertical cooperative learning scheme for web 3.0," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 3983–3994, Sep.–Oct. 2024.

[24] Y. Lu, L. Chen, Y. Wang, and Z. Hu, "Efficient encrypted image content retrieval system based on secureCNN," *Comput. Sci.*, vol. 50, no. 9, pp. 26–34, 2023.

[25] K. Cheng et al., "Manto: A practical and secure inference service of convolutional neural networks for IoT," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14856–14872, Aug. 2023.

[26] A. Stoian, J. Frery, R. Bredehoft, L. Montero, C. Kherfallah, and B. Chevallier-Mames, "Deep neural networks for encrypted inference with TFHE," in *Proc. Int. Symp. Cyber Secur., Cryptol., Mach. Learn.*, 2023, pp. 493–500.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

[27] D. Kim and C. Guyot, "Optimized privacy-preserving CNN inference with fully homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2175–2187, 2023.
[28] D. Kim, J. Park, J. Kim, S. Kim, and J. H. Ahn, "HyPHEN: A hybrid packing method and its optimizations for homomorphic encryption-based neural networks," *IEEE Access*, vol. 12, pp. 3024–3038, 2024.
[29] S. Singh, S. Singh, S. Gudaparthi, X. Fan, and R. Balasubramonian, "Hyena: Balancing packing, reuse, and rotations for encrypted inference," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3091–3108.
[30] E. Lee et al., "Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 12403–12422.
[31] X. Liu, Z. Liu, Q. Li, K. Xu, and M. Xu, "Pencil: Private and extensible collaborative learning without the non-colluding assumption," 2024, *arXiv:2403.11166*.
[32] A. Diaa et al., "Fast and private inference of deep neural networks by co-designing activation functions," in *Proc. 33th USENIX Secur. Symp.*, 2025, pp. 2191–2208.
[33] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "BOLT: Privacy-preserving, accurate and efficient inference for transformers," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 4753–4771.
[34] S. Huang, Y. Zhou, and P. Zheng, "Privacy-preserving inference of binary neural network using fully homomorphic encryption," in *Proc. Int. Conf. Intell. Comput.*, 2025, pp. 235–246.
[35] J. Lin, J. Chen, J. Xiong, D. Jiao, W. Zhao, and Y. Xiang, "A lightweight privacy-preserving federated learning framework for heterogeneity-resilient skin cancer diagnosis," *IEEE J. Biomed. Health Informat.*, early access, Jun. 9, 2025, doi: 10.1109/JBHI.2025.3578125.
[36] H. Ren, X. Lan, R. Tang, Rui, and X. Chen, "PrivDNFIS: Privacy-preserving and efficient deep neuro-fuzzy inference system," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 20174–20182.
[37] D. Liu, W. Yang, C. Peng, N. Wang, R. Hu, and X. Gao, "PrivacyHFR: Visual privacy preserving for heterogeneous face recognition," *IEEE Trans. Image Process.*, vol. 34, pp. 3417–3430, 2025.
[38] M. Zheng, Q. Lou, and L. Jiang, "Primer: Fast private transformer inference on encrypted data," in *Proc. 60th ACM/IEEE Des. Automat. Conf.*, 2023, pp. 1–6.
[39] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2864–2880, Sep.–Oct 2023.
[40] Q. Zhang, C. Xin, and H. Wu, "SecureTrain: An approximation-free and computationally efficient framework for privacy-preserved neural network training," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 187–202, Jan.–Feb 2022.
[41] A. Ali et al., "A novel homomorphic encryption and consortium blockchain-based hybrid deep learning model for industrial Internet of Medical Things," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2402–2418, Sep.–Oct 2023.
[42] C. Hu, J. Liu, H. Xia, S. Deng, and J. Yu, "A lightweight mutual privacy preserving K-Means clustering in industrial IoT," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 2138–2152, Mar.–Apr 2024.
[43] X. Zhou, W. Liang, J. Ma, Z. Yan, and K. I. Wang, "2D federated learning for personalized human activity recognition in cyber-physical-social systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 6, pp. 3934–3944, Nov.–Dec 2022.
[44] J. Pei, Z. Yu, J. Li, M. A. Jan, and K. Lakshmanna, "TKAGFL: A federated communication framework under data heterogeneity," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2651–2661, Sep.–Oct. 2023.
[45] Z. Yuan, Y. Tian, Z. Zhou, T. Li, S. Wang, and J. Xiong, "Trustworthy federated learning against malicious attacks in web 3.0," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 3969–3982, Sep.–Oct. 2024.
[46] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
[47] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th USENIX Secur. Symp.*, 2021, pp. 1251–1263.
[48] J. Liu, M. Juuti, Y. Lu, Yao, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
[49] Z. Ghodsi, A. K. Veldanda, B. Reagen, Brandon, and S. Garg, "CryptoNAS: Private inference on a ReLU budget," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 16961–16971, 2020.
[50] Z. Huang, W. Lu, C. Hong, Cheng, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 809–826.
[51] D. Rathee et al., "SiRnn: A math library for secure RNN inference," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1003–1020.
[52] M. U. Rehman et al., "A novel chaos-based privacy-preserving deep learning model for cancer diagnosis," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 6, pp. 4322–4337, Nov.–Dec. 2022.
[53] J. Wang, D. He, A. Castiglione, B. B. Gupta, M. Karuppiah, and L. Wu, "PCNNCEC: Efficient and privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2906–2923, Sep.–Oct. 2023.
[54] R. Zhao, Y. Xie, D. He, K-K. R. Choo, and Z. L. Jiang, "Practical privacy-preserving convolutional neural network inference framework with edge computing for health monitoring," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5995–6006, Nov.–Dec. 2024.
[55] M. Shen et al., "Secure decentralized aggregation to prevent membership privacy leakage in edge-based federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 3105–3119, May–Jun. 2024.
[56] K. Guo et al., "Privacy-preserving multi-label propagation based on federated learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 886–899, Jan.–Feb. 2024.
[57] Y. Liu et al., "A blockchain-empowered federated learning in healthcare-based cyber physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2685–2696, Sep.–Oct. 2023.
[58] Z. A. EI, A. S. Houda, Hafid, and L. Khoukhi, "MiTFed: A privacy preserving collaborative network attack mitigation framework based on federated learning using SDN and blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1985–2001, Jul.–Aug. 2023.
[59] D. Li et al., "SecDV: A lightweight secure deep neural network inference service with dynamic verification," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 1647–1661, 2026
[60] A. Brüggemann, O. Schick, T. Schneider, A. Suresh, and H. Yalame, "Don't eject the impostor: Fast three-party computation with a known cheater," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 503–522.
[61] F. Liu, X. Xie, and Y. Yu, "Scalable multi-party computation protocols for machine learning in the honest-majority setting," in *Proc. 33th USENIX Secur. Symp.*, 2024, pp. 1939–1956.
[62] B. Yuan, S. Yang, Y. Zhang, N. Ding, D. Gu, and S. F. Sun, "MD-ML: Super fast privacy-preserving machine learning for malicious security with a dishonest majority," in *Proc. 33th USENIX Secur. Symp.*, 2024, pp. 2227–2244.
[63] Y. Dong et al., "PUMA: Secure inference of LLaMA-7B in five minutes," 2023, *arXiv:2307.12533*.
[64] X. Yang, J. Chen, K. He, H. Bai, C. Wu, and R. Du, "Efficient privacy-preserving inference outsourcing for convolutional neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4815–4829, 2023.
[65] W. Zeng et al., "MPCViT: Searching for accurate and efficient mpc-friendly vision transformer with heterogeneous attention," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 5052–5063.
[66] L. Zhou, Z. Wang, H. Cui, Q. Song, and Y. Yu, "Bicoptor: Two-round secure three-party non-linear computation without preprocessing for privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 534–551.
[67] W. Zheng, R. Deng, W. Chen, R. A. Popa, A. Panda, and I. Stoica, "Cerebro: A platform for multi-party cryptographic collaborative learning," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2723–2740.
[68] Y. Chen, Z. Tang, T. Lu, B. Zhang, Z. Shi, and Z. Wang, "Accelerating private large transformers inference through fine-grained collaborative computation," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 7482–7497, 2024.
[69] M. Tong et al., "InferDPT: Privacy-preserving inference for black-box large language models," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 4625–4640, Sep.–Oct. 2025.
[70] J. He et al., "Advancing non-intrusive load monitoring: Predicting appliance-level power consumption with indirect supervision," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2957–2973, Jul.–Aug. 2025.
[71] Z. Chen, L. Zhu, P. Jiang, J. He, and Z. Zhang, "Tackling data mining risks: A tripartite covert channel merging blockchain and IPFS," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 3, pp. 1831–1848, May–Jun. 2025.
[72] J. Ma et al., "SecretFlow-SPU: A performant and user-friendly framework for privacy-preserving machine learning," in *Proc. USENIX Annu. Tech. Conf.*, 2023, pp. 17–33.
[73] J. L. Watson, S. Wagh, and R. A. Popa, "Piranha: A GPU platform for secure computation," in *Proc. 31th USENIX Secur. Symp.*, 2022, pp. 827–844.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.

[74] H. Chen, K. Laine, and R. Player, "Simple encrypted arithmetic library-SEAL v2. 1," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2017, pp. 3–18.
[75] M. Albrecht et al., "Homomorphic encryption standard," in *Protecting Privacy Through Homomorphic Encryption*. Cham, Switzerland: Springer, 2021, pp. 31–62.
[76] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. IEEE Symp. Scurity Privacy*, 2020, pp. 44–55.
[77] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Proc. 29th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2010, pp. 1–23.
[78] Y. LeCun, "The MNIST database of handwritten digits," 1998. [Online]. Available: http://yann.lecun.com.exdb.mnist/
[79] K. Alex, G. Hinton, and N. Vinod, "CIFAR10," 2009. Canadian Institute for Advanced Research [Online]. Available: https://www.cs.toronto.edu/kriz/cifar.html
[80] M. O. Alberto and G. Laurent, "Pyfhel: Python for homomorphic encryption libraries," 2018. [Online]. Available: https://github.com/ibarrond/Pyfhel
[81] D. R. Brown, "Sec 2: Recommended elliptic curve domain parameters," Standards for Efficient Cryptography, 2010. [Online] Avaliable: https://ci.nii.ac.jp/naid/10027922258/en
[82] F. Tramer and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–19.



Dong Li (Graduate Student Member, IEEE) received the M.S. degree in signal and information processing from Southwest University, Chongqing, China, in 2021. He is currently working toward the Ph.D. degree in computer science and technology with Chongqing University, Chongqing. He is also a Visiting Scholar with Nanyang Technological University, Singapore. His research interests include searchable encryption, privacy-preserving, and machine learning.



Anupam Chattopadhyay (Senior Member, IEEE) received the B.E. degree from Jadavpur University, Kolkata, India, in 2000, the M.Sc. degree from ALaRI, Lugano, Switzerland, in 2002, and the Ph.D. degree from RWTH Aachen, Aachen, Germany, in 2008. He held visiting positions at Politecnico di Torino, Turin, Italy; EPFL, Lausanne, Switzerland, Technion, Haifa, Israel, Indian Statistical Institute, Kolkata, and Kyoto University, Kyoto, Japan. In 2014, he was appointed as an Assistant Professor with Nanyang Technological University, Singapore, where he was promoted to Associate Professor with Tenure in 2019. His research interests interests include cyber security, computer architecture, electronic design automation, and emerging technologies. He is also a fellow of JSPS and Inter-Continental Academia and a Senior Member of ACM.



Qianyu Li received the Ph.D. degree in cyberspace security from the National University of Defense Technology, Changsha, China, in 2024. She is currently a Postdoctoral researcher with the National University of Defense Technology. Her research interests include network security and AI security.



Qingguo Lü (Member, IEEE) received the M.S. and Ph.D. degree in signal and information processing and computational intelligence and information processing from Southwest University, Chongqing, China, in 2018 and 2021, respectively. He is currently a Postdoctor and a Hongshen Young Teacher (Special Support) with the College of Computer Science, Chongqing University, Chongqing, China. His research interests include privacy protection, distributed optimization, machine learning, and smart grids. Dr. Lü was the recipient of the Outstanding Ph.D. Thesis Award of ACM China (Chongqing Branch) in 2021, and the Outstanding Master's Thesis Award of Chongqing, in 2019. He is also a Guest Editor of *Applied Sciences (SI: Advanced Artificial Intelligence Theories and Applications*.



Jiahui Wu received the Ph.D. degrees in signal and information processing and computational intelligence and information processing from Southwest University, Chongqing, China, in 2021, respectively. She is currently an Assistant Researcher with Peng Cheng Laboratory, Shenzhen, China. Her research interests include information security, cloud computing security, and data mining.



Tao Xiang (Senior Member, IEEE) received the B.Eng., M.S. and Ph.D. degrees in computer science from Chongqing University, Chongqing, China, in 2003, 2005, and 2008, respectively. He is currently a Professor with the College of Computer Science, Chongqing University. His research interests include multimedia security, cloud security, data privacy and cryptography. He has authored or coauthored more than 100 papers on international journals and conferences. He was also a Referee for numerous international journals and conferences.



Xiaofeng Liao (Fellow, IEEE) received the B.S. and M.S. degrees in mathematics from Sichuan University, Chengdu, China, in 1986 and 1992, respectively, and the Ph.D. degree in circuits and systems from the University of Electronic Science and Technology of China, Chengdu, China, in 1997. From 1999 to 2012, he was a Professor with Chongqing University, Chongqing, China. From 2012 to 2018, he was a Professor and the Dean with the College of Electronic and Information Engineering, Southwest University, Chongqing. He is currently a Professor and the Dean with the College of Computer Science, Chongqing University. He is also a Yangtze River Scholar with the Ministry of Education of China, Beijing, China. From 1997 to 1998, he was a Research Associate with the Chinese University of Hong Kong, Hong Kong. From 1999 to 2000, he was a Research Associate with the City University of Hong Kong, Hong Kong. From 2001 to 2001 and from 2002 to 2002, respectively, he was a Senior Research Associate wirth the City University of Hong Kong. From 2006 to 2007, he was a Research Fellow with the City University of Hong Kong. He also holds five patents, and published four books and more than 300 international journal and conference papers. His research interests include decentralized optimization, machine learning, neural networks, and bifurcation and chaos cryptography. He is also an Associate Editor for IEEE TRANSACTIONS ON CYBERNETICS and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on October 11,2026 at 12:19:27 UTC from IEEE Xplore. Restrictions apply.